Recall → Recovery E2E evidence

Run: 20260610T113622Z **Scope:** FAILURE → operator-RECALL(forward-fix) → device-RECOVERY on the Tier-1 emulator, proven in-process over real HTTP (httptest + real `api.Server` + real `deviceemu.Device`).

Test

`server/internal/deviceemu/recall_recovery_test.go` — `TestRecallRecoveryE2E`.

Commands run + results

Command	Result
`cd server && go test -v -run RecallRecovery ./internal/deviceemu/...`	**PASS** (exit 0) — see `go_test_v_transcript.txt`
`cd server && go test -race -v -run RecallRecovery ./internal/deviceemu/...`	**PASS** (exit 0, no data race) — see `go_test_race_transcript.txt`
`cd server && go test -race ./internal/deviceemu/...` (whole package)	**PASS** (no interaction breakage with the existing lifecycle tests)

Asserted steps (every one against real response data)

1. **Stage v1.1.0 + deployment D1** — operator uploads a signed artifact, publishes release `1.1.0`, creates an all-targets deployment D1.
2. **Apply** — device (current `1.0.0`) RunOnce → 200 offer of `1.1.0` carrying D1 → applies → version advances to `1.1.0`, healthy=true, telemetry accepted (rejected=0). Operator cross-check `GET /devices/{id}/status` shows `1.1.0` + healthy.
3. **Failure** — `ReportFailure(ctx,"post_apply_health_check_failed")`: telemetry accepted=1 rejected=0, device healthy=false, version unchanged (`1.1.0`). Server cross-checks: `/status` health.ok=false; `/telemetry?event=failure` shows the failure stamped with D1 + the error code.
4. **Forward-fix recall** — operator stages release `1.2.0` (release only) and `POST /deployments/{D1}/recall` with `to_release_id`=the `1.2.0` release → **201**, kind=rollback, from=`1.1.0` release, to=`1.2.0` release, details.mode=forward-fix, non-empty recall_deployment_id. `GET /deployments/{D1}/rollbacks` confirms the row.

5. **Recovery** — device RunOnce → 200 offer of `1.2.0` carrying the NEW recall deployment id → applies → version advances to `1.2.0`, healthy=true again, telemetry accepted (rejected=0). Server cross-checks: `/status` health.ok=true + `1.2.0`; `/telemetry?event=success` stamped with the recall deployment id.
6. **On-target** — a third RunOnce → 204 on-target (applied=false).

Finding surfaced during authoring (anti-bluff)

Staging a second all-targets deployment for the fix release while D1 is still active returns **409 CONFLICT** ("an active deployment already targets this set") — correct server behaviour. The recall endpoint itself supersedes D1 and creates the recall deployment, so the fix is staged as a **release only**. This matched the recall handler's documented forward-fix design.

Container sibling

`tests/emulator/tier1_recall_recovery_e2e.sh` is the podman variant of this flow. It is `bash -n` and `sh -n` clean (§11.4.67) but was NOT run live during this parallel phase (podman may be in use by a concurrent agent). The conductor must run the live podman variant.